# All UML Diagrams – PolyFab ERP System

**For more information: view SRS Document from this google drive only**

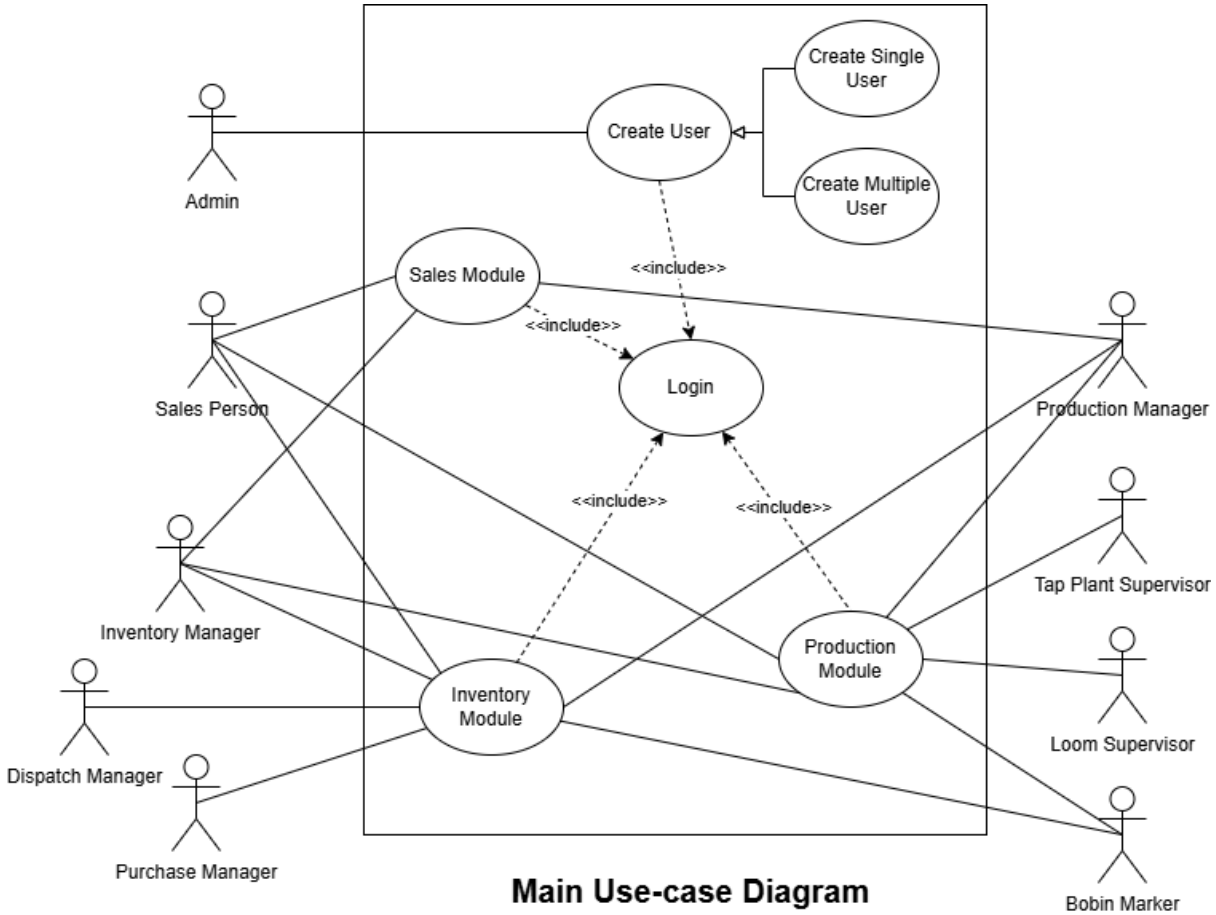
## Prepared by:

**Niraj Chaudhary : 22BCE209**

**Ankit Pandor : 22BCE218**

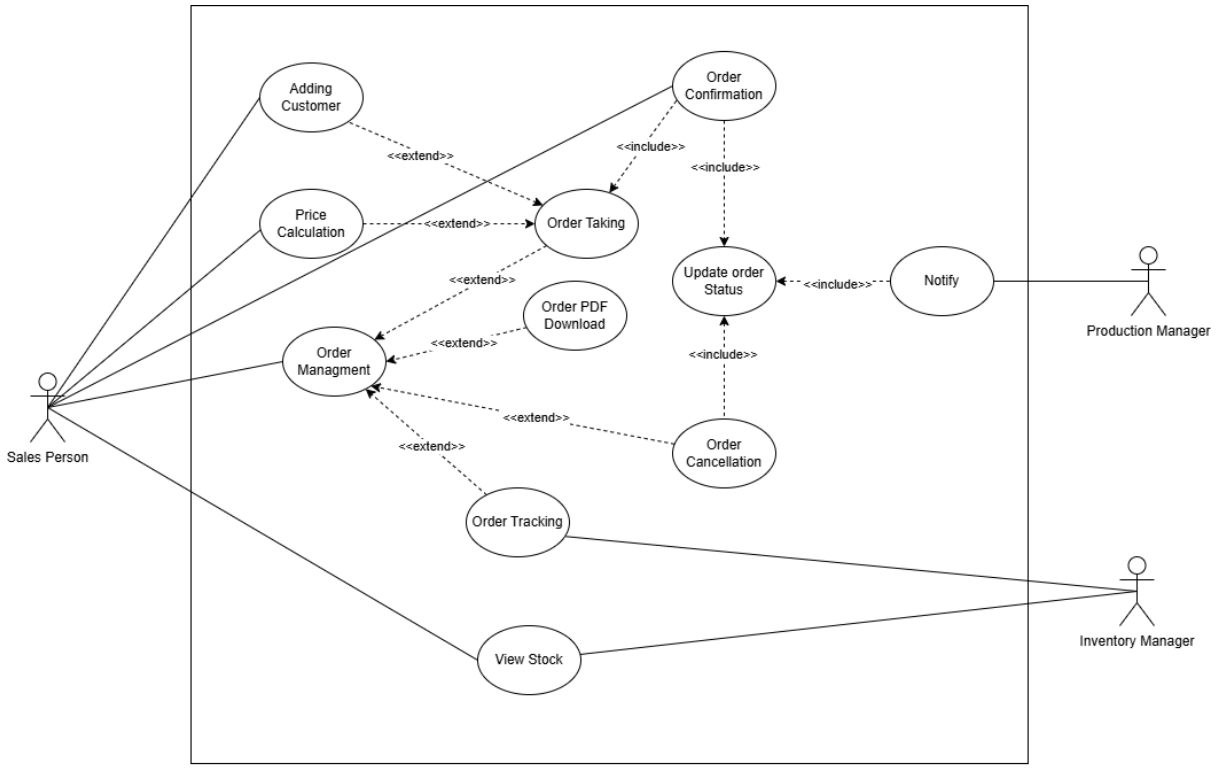
**Samarth Pansala : 22BCE220**

## Use Case Diagrams:



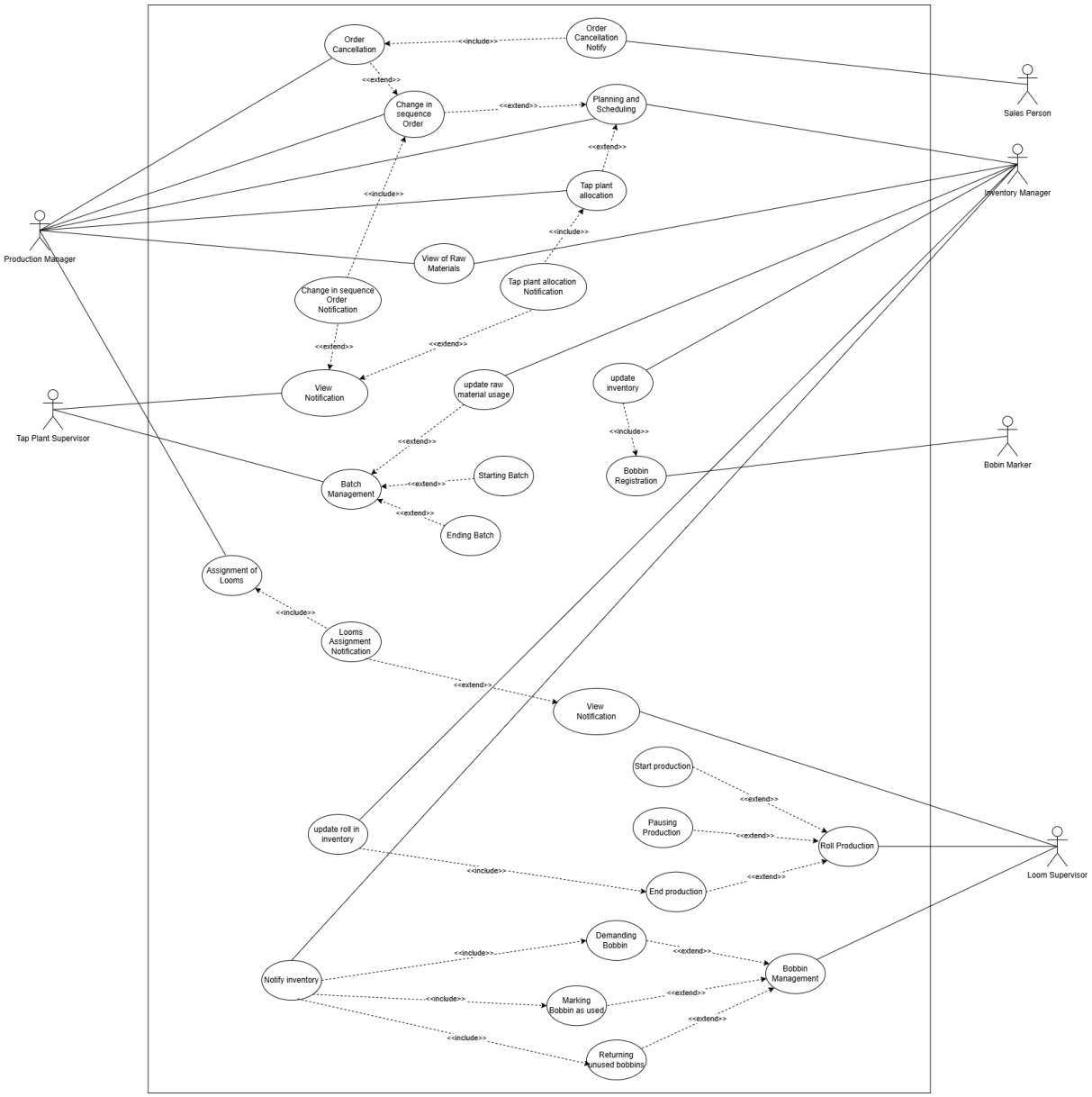

Main Use-case Diagram

Samarth

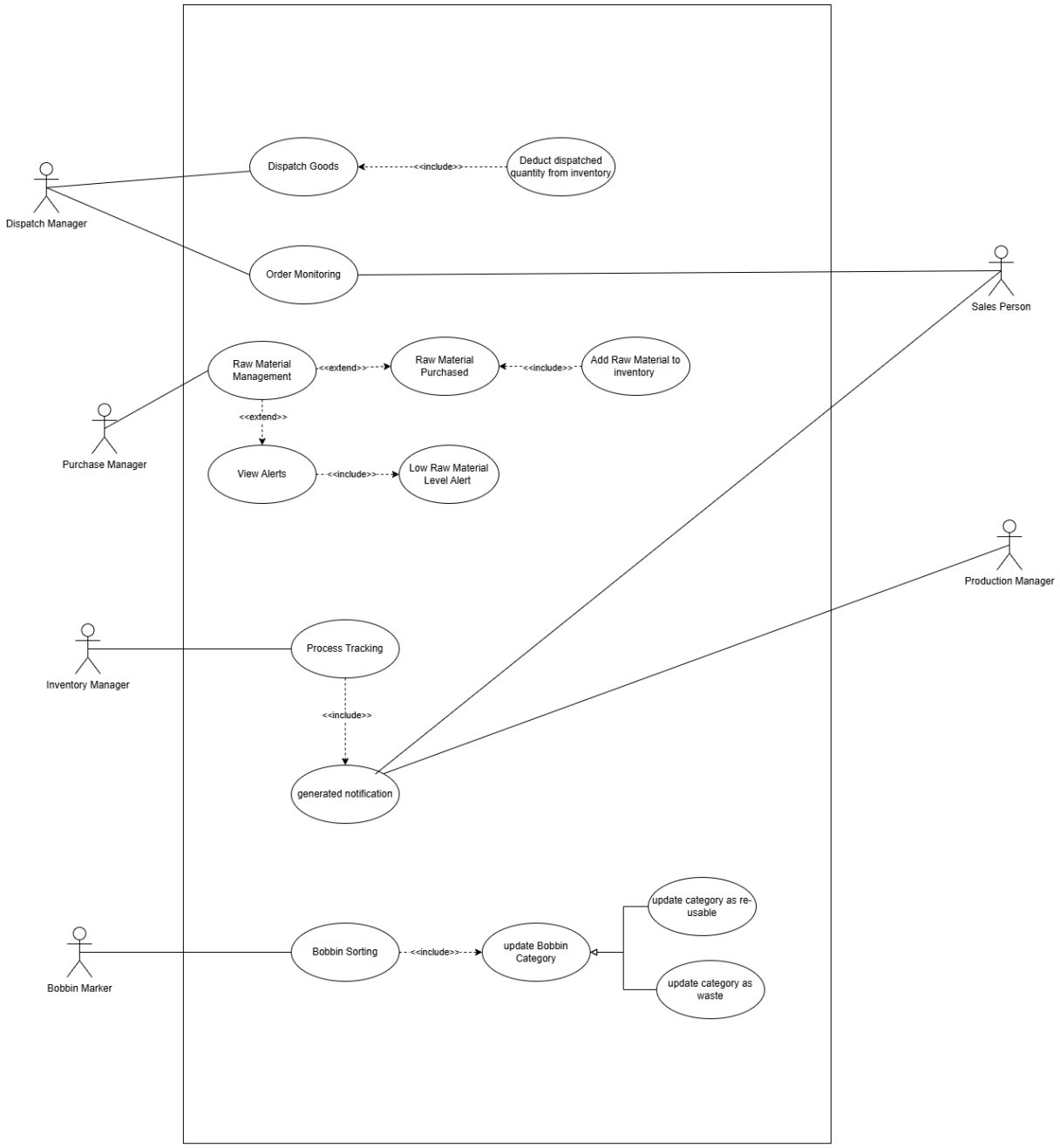



**Sales Module Use-case Diagram**

Samarth

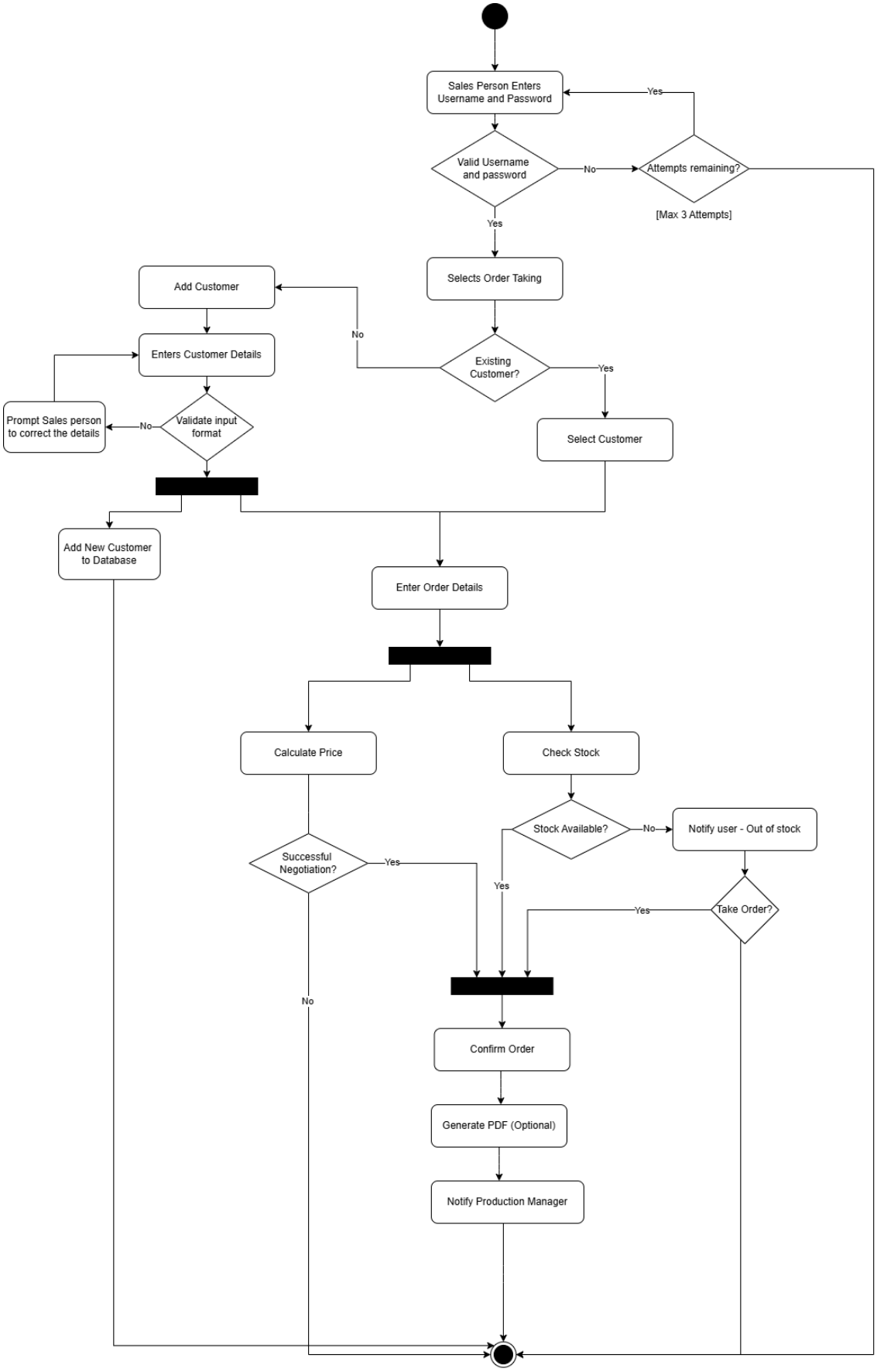



**Production Module Use-case Diagram**

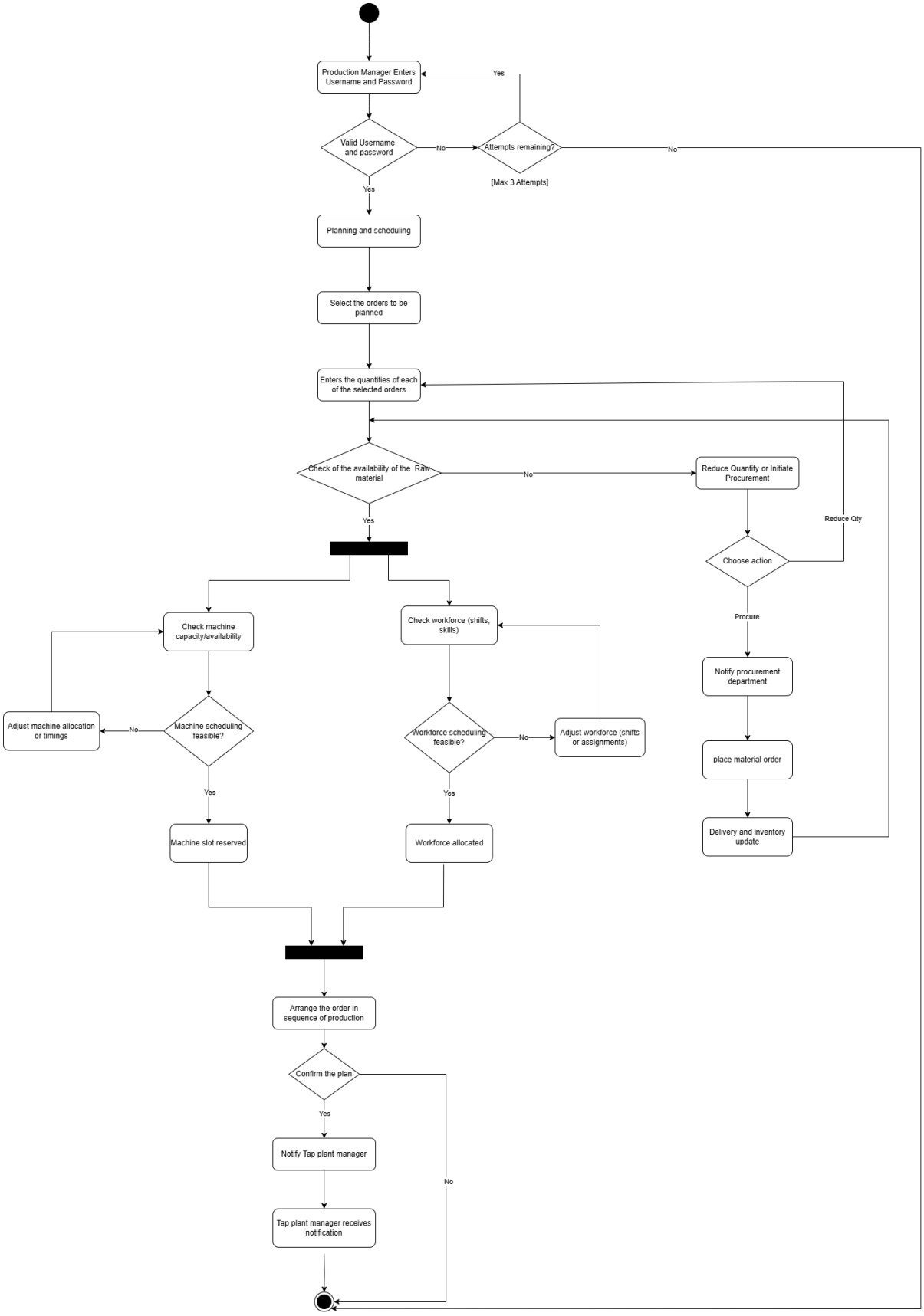



**Inventory Module Use-case Diagram**

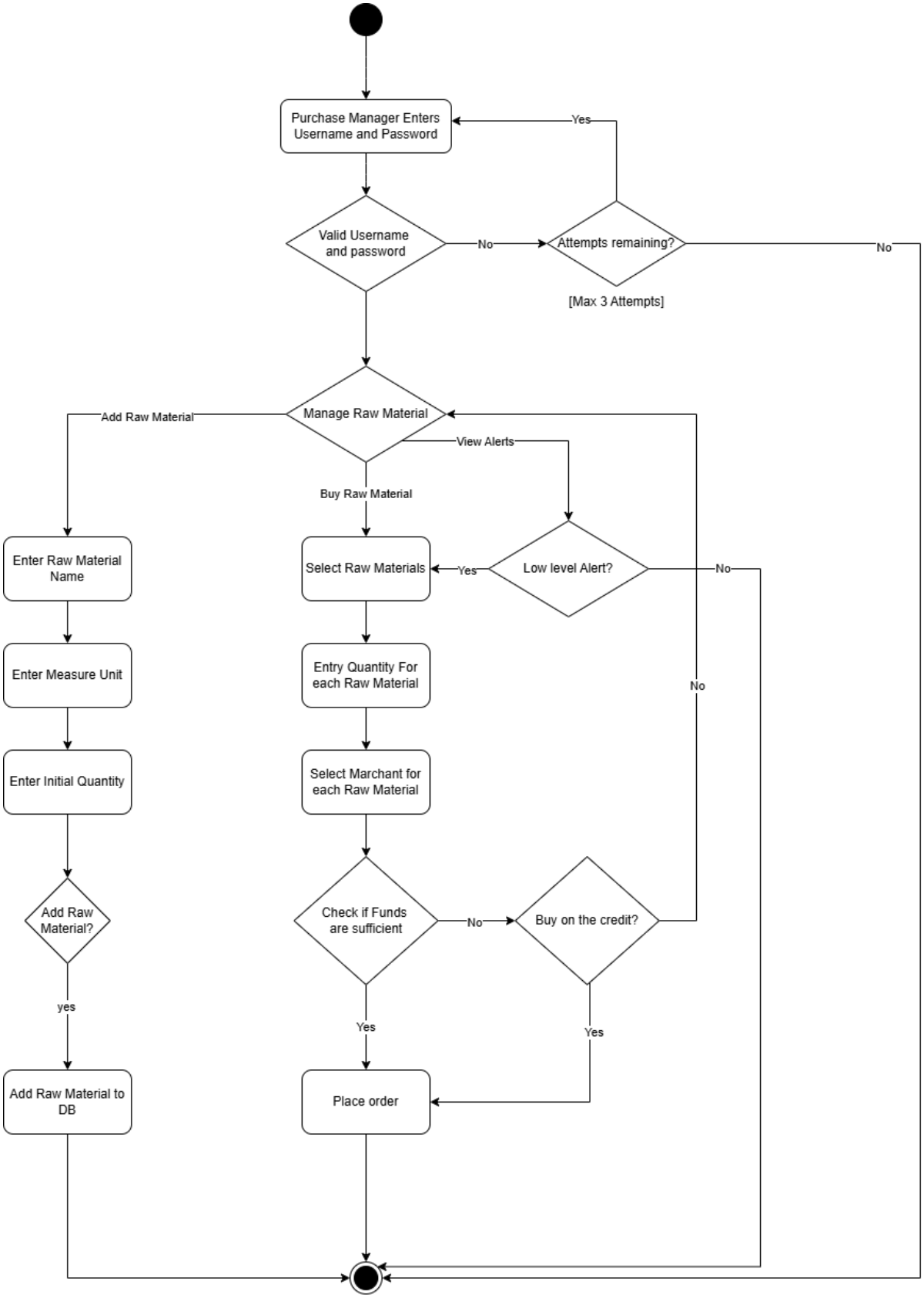
# Activity Diagrams:




**Activity Diagram for sales module**

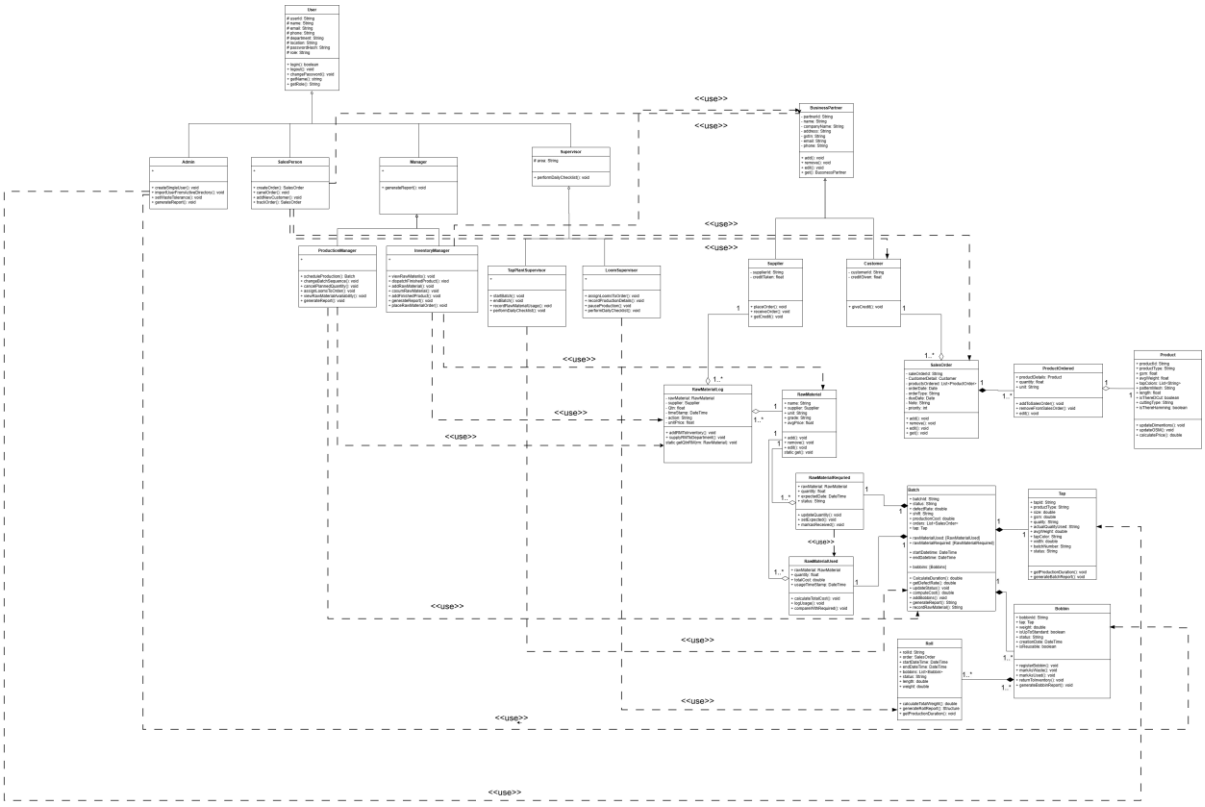



**Activity Diagram for production module**



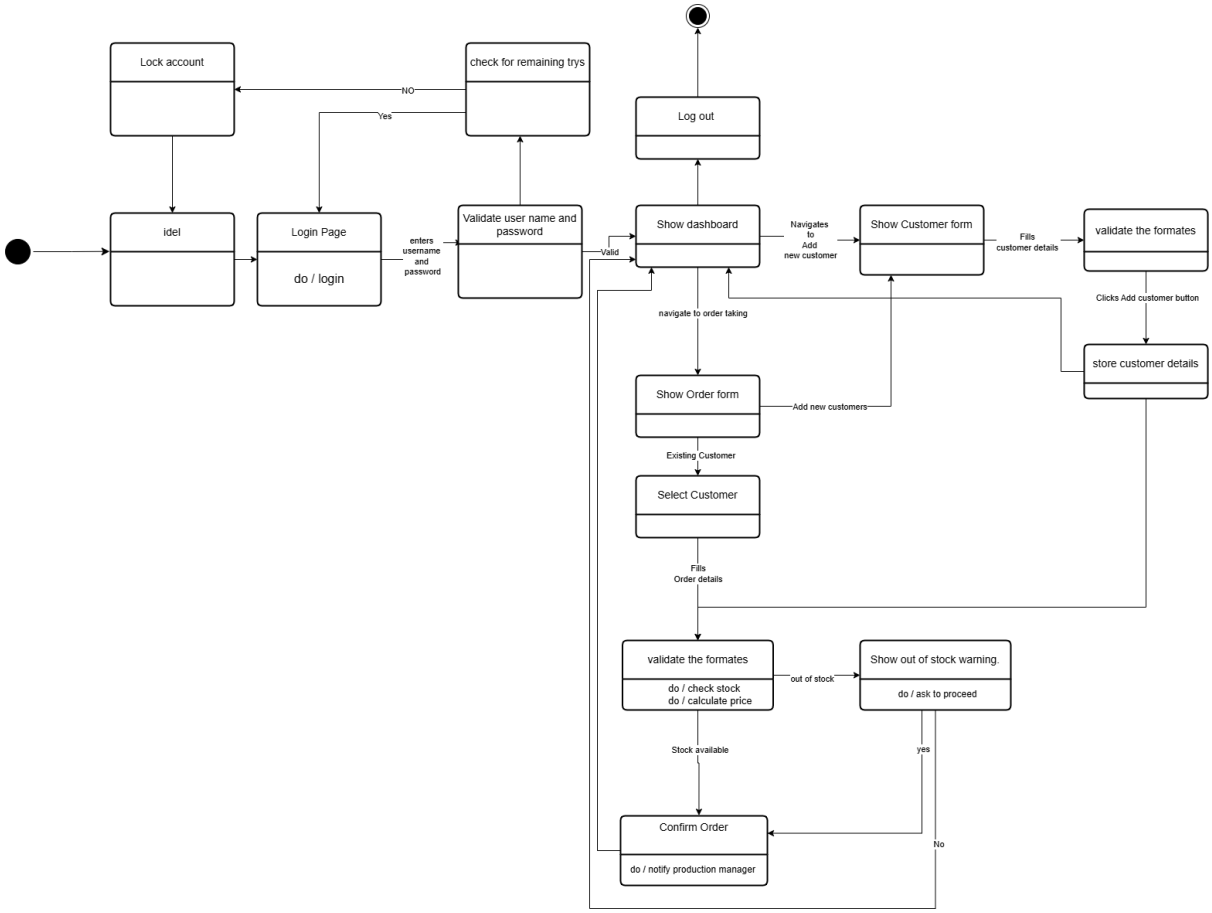

**Activity Diagram for inventory module**

# Class Diagram:

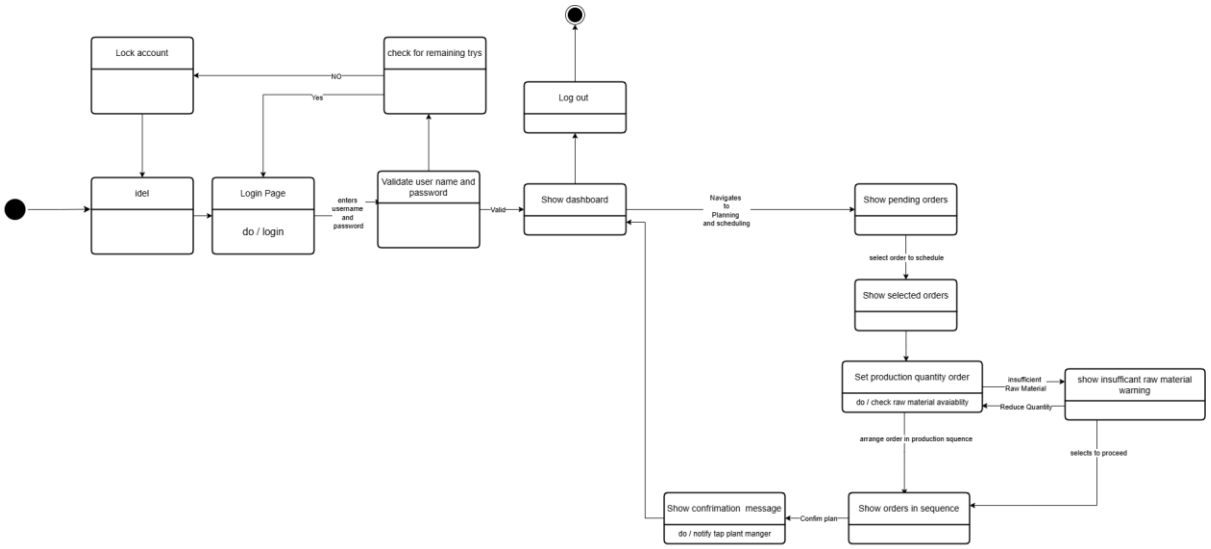



# CRC Index Cards:

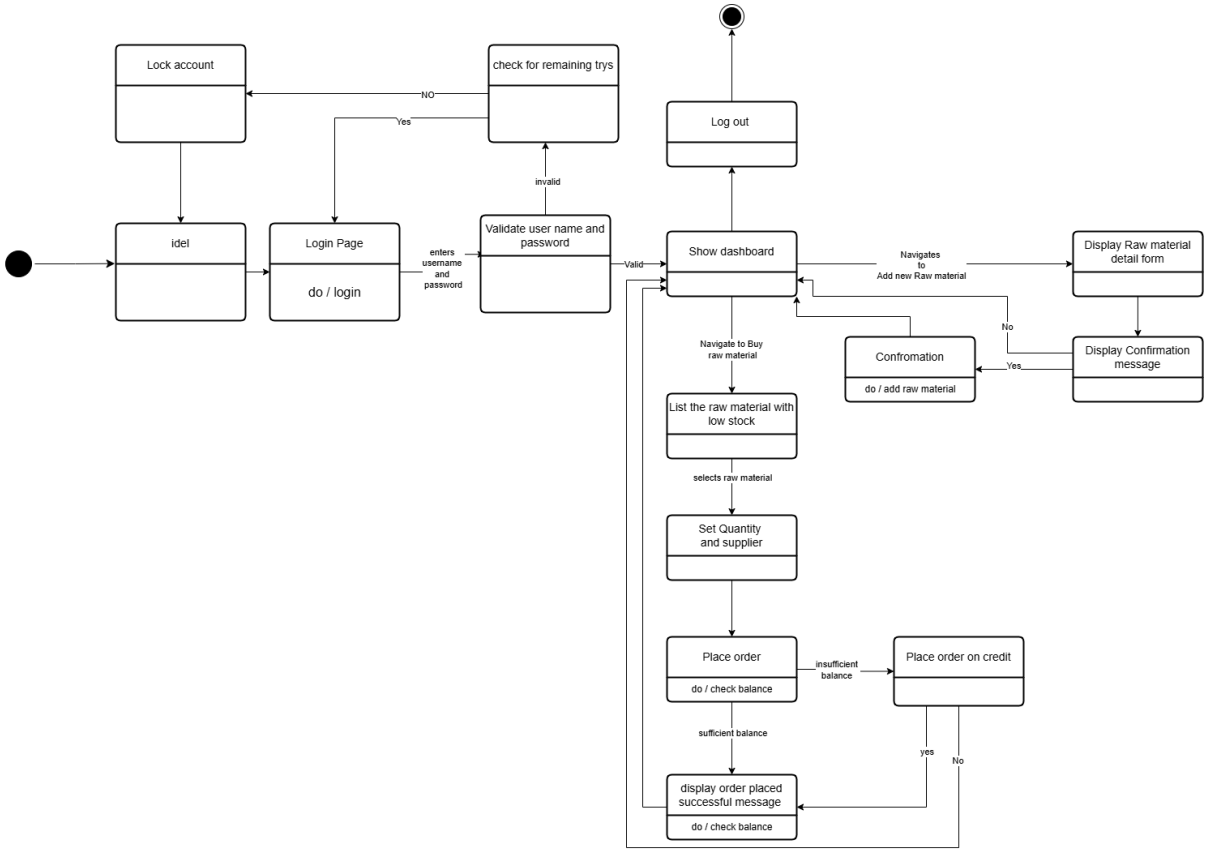
# State Diagrams:




**State Diagram - Sales Module**

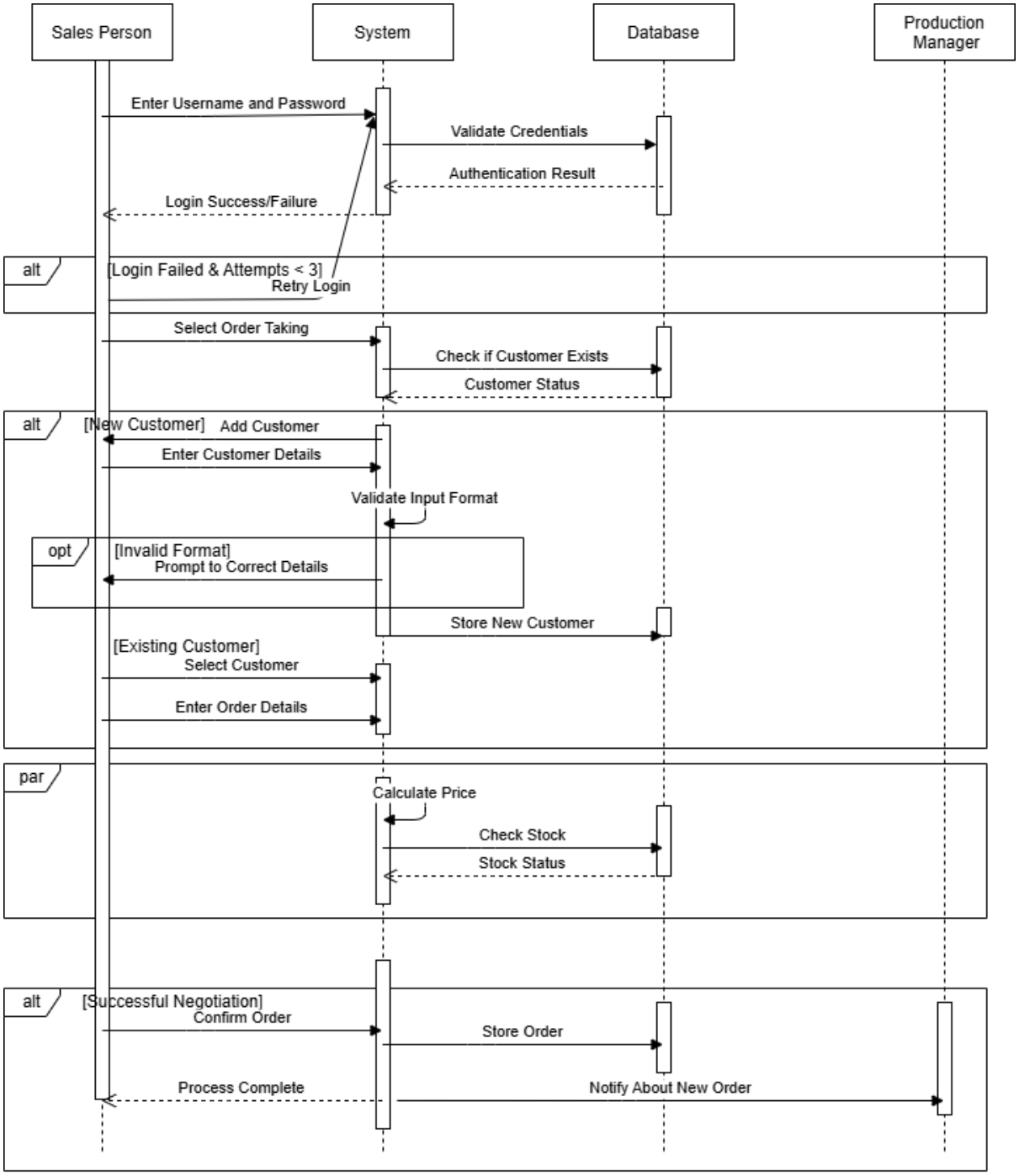



**State Diagram - Production Module**

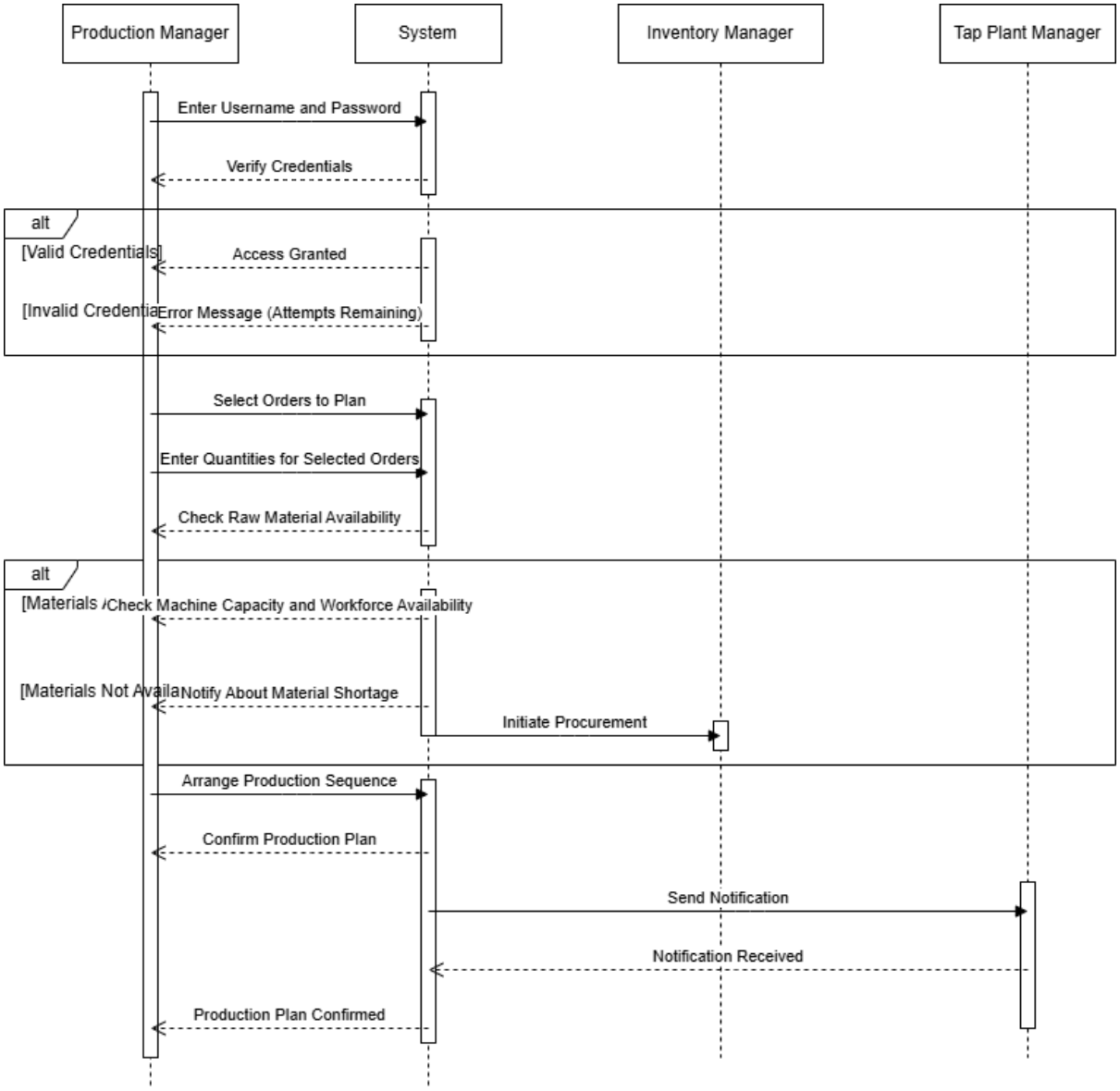



**State Diagram - Inventory Module**

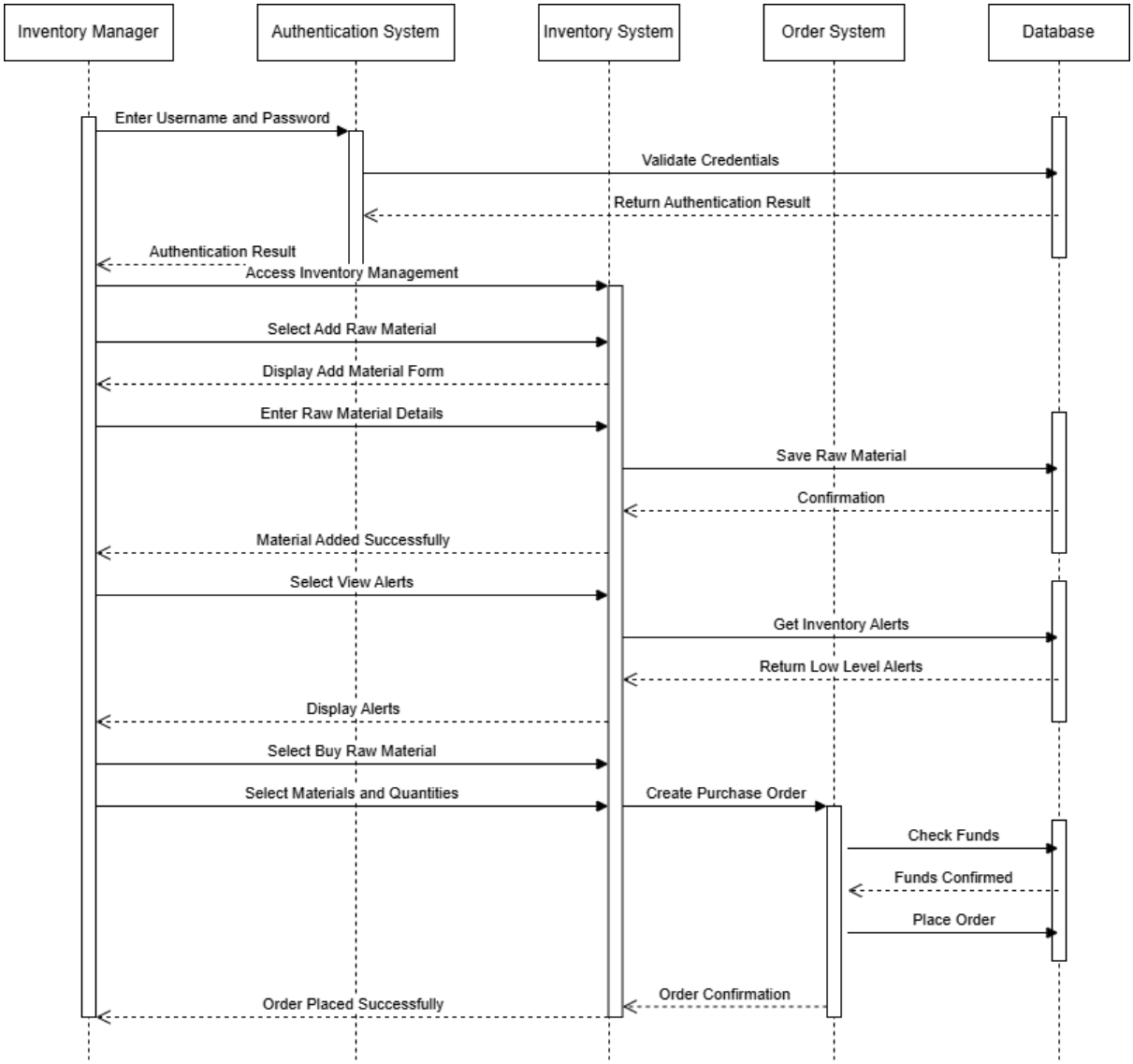
# Sequence Diagrams:




# Sequence Diagram - Sales Module

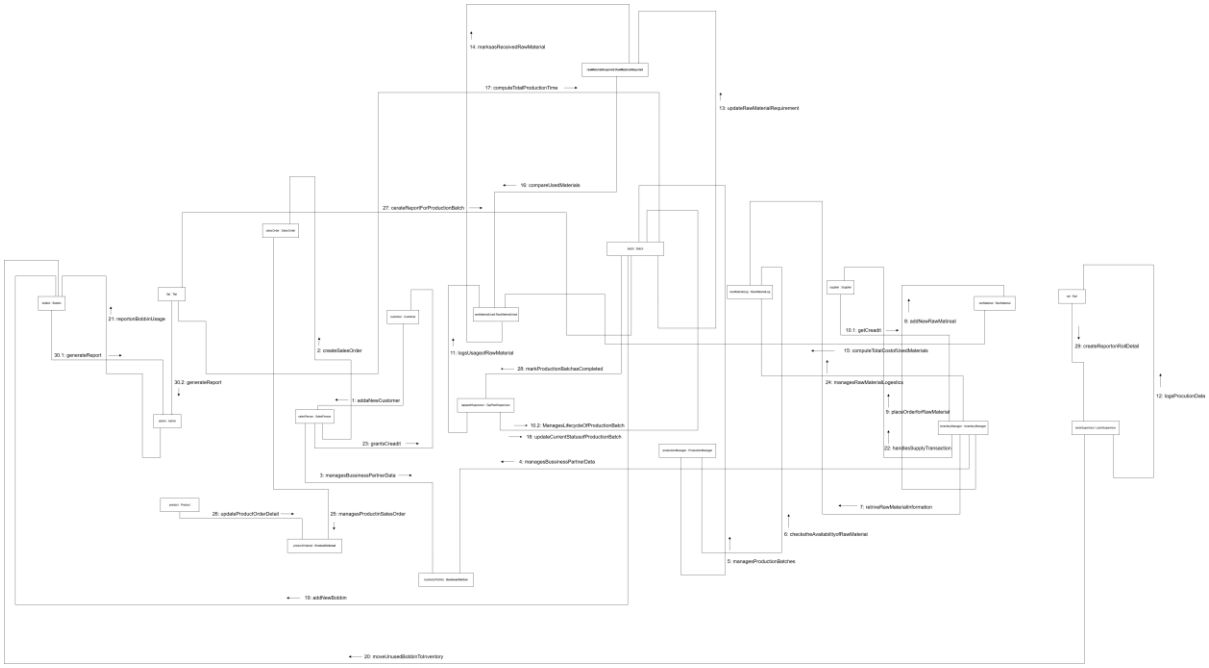



## Sequence Diagram - Production Module




**Sequence Diagram - Inventory Module**

# Collaboration Diagram:




# Formal Specifications:

## Formal Specification - User Management Module

User

uid: USERID
name: NAME
role: ROLE
email: EMAIL
phone: PHONE
dept: DEPARTMENT
loc: LOCATION

[USER TYPES]

USER ::= Admin | SalesPerson | Manager | Supervisor

MANAGER ::= ProductionManager | PurchaseManager | DispatchManager | InventoryManager

SUPERVISOR ::= TapPlantSupervisor | LoomSupervisor | FinishingSupervisor

AddUser

Δ UserDB
uid?: USERID
name?: NAME
role?: ROLE
email?: EMAIL
phone?: PHONE
dept?: DEPARTMENT
loc?: LOCATION

---

Precondition:
uid? ∉ dom users

Postcondition:
users' = users ∪ { uid? ↦(name?, role?,
email?, phone?,
dept?, loc?) }

LoginUser

Ξ UserDB
email?: EMAIL
password?: PASSWORD
uid!: USERID
authenticated!: BOOLEAN

---

Precondition:
∃ u: USERID | u ∈ dom users ∧
(email? = (users u).email)

Postcondition:
(∃ u: USERID | u ∈ dom users ∧
(email? = (users u).email) ∧
validatePassword(password?, u)) ⇒
(authenticated! = TRUE ∧ uid! = u)
∧
(¬ ∃ u: USERID | u ∈ dom users ∧
(email? = (users u).email) ∧
validatePassword(password?, u)) ⇒
(authenticated! = FALSE)

## Formal Specification - Sales Module

[ORDER]
orderId: ORDERID
orderType: ORDERTYPE
customerId: CUSTOMERID
product: PRODUCT
quantity: QUANTITY
dueDate: DATE
note: NOTE
status: STATUS
salespersonId: USERID

Customer

customerId: CUSTOMERID
companyName: NAME
contactPerson: NAME
email: EMAIL
gstin: GSTIN
phone: PHONE

CreateOrder

Δ OrderDB
orderId?: ORDERID
orderType?: ORDERTYPE
customerId?: CUSTOMERID
product?: PRODUCT
quantity?: QUANTITY
dueDate?: DATE
note?: NOTE
salespersonId?: USERID

Precondition:
orderId? ∉ dom orders ∧
(orderType? = 'custom' ∨ orderType? = 'jobwork' ⇒
customerId? ∈ dom customers) ∧
salespersonId? ∈ dom users ∧
(users salespersonId?).role = 'SALESPERSON'

Postcondition:
orders' = orders ∪ { orderId? ↦(orderType?, customerId?,
product?, quantity?, dueDate?, note?,
'pending', salespersonId?) }

AddCustomer

Δ CustomerDB
companyName?: NAME
contactPerson?: NAME
email?: EMAIL
gstin?: GSTIN
phone?: PHONE
customerId!: CUSTOMERID

Precondition:
∀ c : dom customers • (customers c).email ≠ email? ∧
(customers c).gstin ≠ gstin?

Postcondition:
customerId! ∉ dom customers ∧
customers' = customers ∪ { customerId! ↦(companyName?,
contactPerson?, email?, gstin?, phone?) }

CancelOrder

Δ OrderDB
orderId?: ORDERID
cancelReason?: REASON

Precondition:
orderId? ∈ dom orders ∧
(orders orderId?).status ∈ {'pending', 'inProduction'}

Postcondition:
orders' = orders ⊕ { orderId? ↦
(orders orderId?) ⊕
{status ↦ 'cancelled',
cancelReason ↦ cancelReason?} } ∧
(∀ l : LOOMID | l ∈ dom looms ∧ (looms l).orderId =
orderId? •
looms' = looms ⊕ { l ↦ (looms l) ⊕ {status ↦ 'available',
orderId ↦ ∅} })

Samarth

# Formal Specification - Production Module

### Batch

batchId: BATCHID
orderId: ORDERID
status: STATUS
startTime: DATETIME
endTime: DATETIME
product: PRODUCT
quantity: QUANTITY
wastage: QUANTITY

### Bobbin

bobbinId: BOBBINID
weight: WEIGHT
status: STATUS
creationDate: DATETIME
batchId: BATCHID
materialType: MATERIALTYPE

### Loom

loomId: LOOMID
status: STATUS
orderId: ORDERID
assignedWorker: USERID
startTime: DATETIME
endTime: DATETIME
currentRoll: ROLLID

### StartBatch

Δ BatchDB
Δ InventoryDB
batchId?: BATCHID

Precondition:
batchId? ∈ dom batches ∧
(batches batchId?).status = 'scheduled' ∧
checkRawMaterialAvailability((batches batchId?).product)

Postcondition:
batches' = batches ⊕ { batchId? ↦
(batches batchId?) ⊕ {status ↦ 'inProgress', startTime ↦
currentDateTime} }

### RegisterBobbin

Δ BobbinDB
weight?: WEIGHT
materialType?: MATERIALTYPE
batchId?: BATCHID
bobbinId!: BOBBINID

Precondition:
weight? > 0 ∧
batchId? ∈ dom batches

Postcondition:
bobbinId! ∉ dom bobbins ∧
bobbins' = bobbins ∪ { bobbinId! ↦(weight?,
'available', currentDateTime, batchId?, materialType?) }

### AssignLooms

Δ LoomDB
Δ OrderDB
orderId?: ORDERID
loomIds?: ℙ LOOMID

Precondition:
orderId? ∈ dom orders ∧
(orders orderId?).status = 'inProduction' ∧
loomIds? ⊆ dom looms ∧
∀ l : loomIds? • (looms l).status = 'available'

Postcondition:
looms' = looms ⊕ { l : loomIds? • l ↦
(looms l) ⊕ {status ↦ 'inUse', orderId ↦ orderId?} } ∧
orders' = orders ⊕ { orderId? ↦
(orders orderId?) ⊕ {assignedLooms ↦
(orders orderId?).assignedLooms ∪ loomIds?} }

### RecordProductionWastage

Δ WastageDB
Δ BatchDB
batchId?: BATCHID
wastageQty?: QUANTITY
wastageType?: WASTAGETYPE
wastageLocation?: LOCATION

Precondition:
batchId? ∈ dom batches ∧
(batches batchId?).status ∈ {'inProgress', 'completed'} ∧
wastageQty? ≥ 0

Postcondition:
Let tolerance == getToleranceLevel(wastageType?, wastageLocation?) •
Let exceedsTolerance == wastageQty? > tolerance •
wastages' = wastages ∪ { nextWastageId ↦
(batchId?, wastageQty?, wastageType?, wastageLocation?,
currentDateTime, exceedsTolerance)} ∧
batches' = batches ⊕ { batchId? ↦
(batches batchId?) ⊕ {wastage ↦ (batches batchId?).wastage +
wastageQty?} }

### AssignBobbinToLoom

Δ BobbinDB
Δ LoomBobbinDB
bobbinId?: BOBBINID
loomId?: LOOMID

Precondition:
bobbinId? ∈ dom bobbins ∧
(bobbins bobbinId?).status = 'available' ∧
loomId? ∈ dom looms ∧
(looms loomId?).status = 'inUse'

Postcondition:
bobbins' = bobbins ⊕ { bobbinId? ↦
(bobbins bobbinId?) ⊕ {status ↦ 'assigned'} } ∧
loomBobbins' = loomBobbins ∪ {(loomId?, bobbinId?, currentDateTime)}

### AssignWorkerToLoom

Δ LoomDB
loomId?: LOOMID
workerId?: USERID

Precondition:
loomId? ∈ dom looms ∧
(looms loomId?).status = 'inUse' ∧
workerId? ∈ dom users ∧
(users workerId?).role = 'WORKER'

Postcondition:
looms' = looms ⊕ { loomId? ↦
(looms loomId?) ⊕ {assignedWorker ↦ workerId?} }

### EndBatch

Δ BatchDB
Δ OrderDB
batchId?: BATCHID
finalQuantity?: QUANTITY
finalWastage?: QUANTITY

Precondition:
batchId? ∈ dom batches ∧
(batches batchId?).status = 'inProgress' ∧
finalQuantity? ≥ 0 ∧
finalWastage? ≥ 0

Postcondition:
Let orderId == (batches batchId?).orderId •
batches' = batches ⊕ { batchId? ↦
(batches batchId?) ⊕
{status ↦ 'completed',
endTime ↦ currentDateTime,
quantity ↦ finalQuantity?,
wastage ↦ finalWastage?} } ∧
orders' = orders ⊕ { orderId ↦
(orders orderId) ⊕
{scheduledQty ↦ (orders orderId).scheduledQty - (batches
batchId?).quantity + finalQuantity?} }

### RequestBobbins

Δ BobbinRequestDB
Ξ LoomDB
loomId?: LOOMID
requiredTypes?: ℙ MATERIALTYPE
requestId!: REQUESTID

Precondition:
loomId? ∈ dom looms ∧
(looms loomId?).status = 'inUse' ∧
requiredTypes? ≠ ∅

Postcondition:
requestId! ∉ dom bobbinRequests ∧
bobbinRequests' = bobbinRequests ∪ { requestId! ↦
(loomId?, requiredTypes?, currentDateTime, 'pending') }

### PauseLoom

Δ LoomDB
loomId?: LOOMID
reason?: REASON

Precondition:
loomId? ∈ dom looms ∧
(looms loomId?).status = 'inUse'

Postcondition:
looms' = looms ⊕ { loomId? ↦
(looms loomId?) ⊕
{status ↦ 'paused',
pauseReason ↦ reason?,
pauseStartTime ↦ currentDateTime} }

### SetWastageToleranceLevel

Δ ToleranceDB
processType?: PROCESSTYPE
location?: LOCATION
tolerancePercentage?: PERCENTAGE

Precondition:
tolerancePercentage? ≥ 0 ∧ tolerancePercentage? ≤ 100

Postcondition:
toleranceLevels' = toleranceLevels ⊕
{(processType?, location?) ↦ tolerancePercentage?}

### GenerateProductionReport

Ξ BatchDB
Ξ OrderDB
Ξ WastageDB
startDate?: DATE
endDate?: DATE
report!: REPORT

Precondition:
startDate? ≤ endDate? ∧ endDate? ≤ currentDate

Postcondition:
Let relevantBatches == { b: dom batches |
(batches b).startTime ≥ startDate? ∧
(batches b).startTime ≤ endDate? } •
Let batchStats == { b: relevantBatches • b ↦
(calculateEfficiency(b), calculateWastagePercentage(b)) } •
report! = compileReport(relevantBatches, batchStats)

**Formal Specification - Inventory Module**

Dispatch

dispatchId: DISPATCHID
orderId: ORDERID
quantity: QUANTITY
dispatchDate: DATETIME
transportDetails: TRANSPORT
status: STATUS

[RAWMATERIAL]
materialId: MATERIALID
name: NAME
quantity: QUANTITY
batchNo: BATCHNO
supplier: SUPPLIER

UpdateInventory

Δ InventoryDB
materialId?: MATERIALID
quantity?: QUANTITY
batchNo?: BATCHNO
supplier?: SUPPLIER

Precondition:
materialId? ∈ dom materials

Postcondition:
materials' = materials ⊕ { materialId? ↦
(materials materialId?) ⊕
{quantity ↦ (materials materialId?).quantity + quantity?} }

CreateDispatch

Δ DispatchDB
Δ OrderDB
Δ InventoryDB
orderId?: ORDERID
quantity?: QUANTITY
transportDetails?: TRANSPORT
dispatchId!: DISPATCHID

Precondition:
orderId? ∈ dom orders ∧
(orders orderId?).status ∈ {'completed', 'partiallyDispatched'} ∧
quantity? > 0 ∧
quantity? ≤ ((orders orderId?).quantity - (orders orderId?).dispatchedQty) ∧
checkInventoryAvailability((orders orderId?).product, quantity?)

Postcondition:
dispatchId! ∉ dom dispatches ∧
dispatches' = dispatches ∪ { dispatchId! ↦
(orderId?, quantity?, currentDateTime, transportDetails?, 'dispatched') } ∧
Let newDispatchedQty == (orders orderId?).dispatchedQty + quantity? •
Let newStatus == (newDispatchedQty = (orders orderId?).quantity ? 'fullyDispatched' :
'partiallyDispatched') •
orders' = orders ⊕ { orderId? ↦
(orders orderId?) ⊕
{dispatchedQty ↦ newDispatchedQty, status ↦ newStatus} } ∧
inventory' = updateInventory(inventory, (orders orderId?).product, -quantity?)

DispatchOrder

Δ OrderDB
Δ InventoryDB
orderId?: ORDERID
dispatchQuantity?: QUANTITY

Precondition:
orderId? ∈ dom orders ∧
(orders orderId?).status = 'completed' ∧
dispatchQuantity? > 0 ∧
dispatchQuantity? ≤ (orders orderId?).pendingQuantity ∧
(getInventoryForProduct((orders orderId?).product) ≥
dispatchQuantity?)

Postcondition:
Let pendingQty == (orders orderId?).pendingQuantity -
dispatchQuantity? •
orders' = orders ⊕ { orderId? ↦
(orders orderId?) ⊕
{pendingQuantity ↦ pendingQty,
dispatchedQuantity ↦ (orders orderId?).dispatchedQuantity +
dispatchQuantity?,
status ↦ (pendingQty = 0 ? 'dispatched' : 'partiallyDispatched')} }
∧
inventory' = updateInventory(inventory, (orders orderId?).product,
-dispatchQuantity?)